RLVP: Penalize the Path, Reward the Outcome

Bojie Li
Pine AI

Noah Shi
University of Washington

Abstract

Agents that act in the world on our behalf—placing phone calls, resolving support tickets—learn *online*, where every rollout is a real, often irreversible interaction, not a cheap simulator step. Two things follow. First, whether such an agent is *deployable* is decided by its path, not its outcome: it must not keep calling a user who has not answered, act outside business hours, or skip a service's required authentication—constraints the outcome reward cannot express, since doing more of the forbidden thing often *improves* the outcome. Second, because each interaction is costly, it must learn from few of them.

Reinforcement learning from verifiable rewards is blind to both: it optimizes only the outcome, and wastes expensive rollouts on the all-failing groups where a group-relative advantage is zero. Going denser by rewarding *progress* reaches for the hard-to-verify direction; agentic environments instead verify a *bad* move cheaply and progress not at all. Because a group-relative advantage *is* a within-group variance, a dense signal helps only where it supplies variance the outcome lacks. A verifiable *penalty* on the path meets this by construction; a progress *potential* only where partial progress is reachable, turning wasted rollouts into learning there. The recipe—**penalize the path, reward the outcome**—attains the task with near-zero violations where outcome-only training violates every episode, and cuts harmful actions *sixfold* at equal success; the potential is its sample-efficiency mirror. We give four design rules, including that a lone penalty stalls in an *inaction trap*.

Code: https://github.com/19PINE-AI/rlvp
Website: https://01.me/research/rlvp

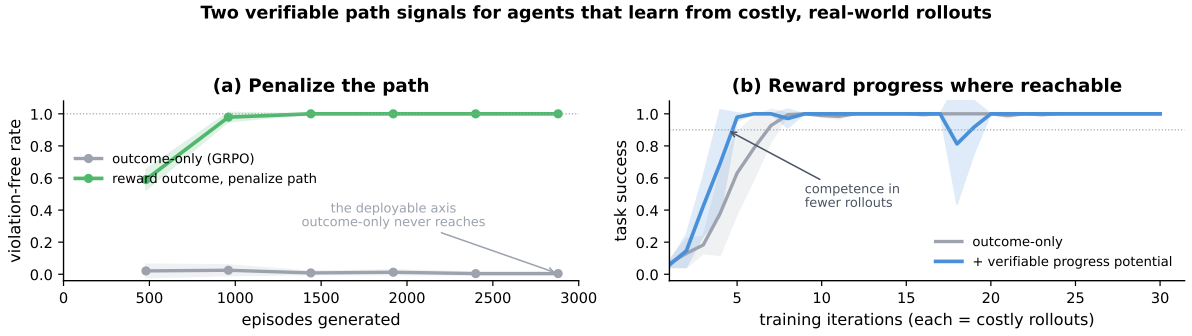

Figure 1. Two verifiable path signals for agents that learn from costly, real-world rollouts. *(a) Penalize the path.* On diverse held-out tasks, rewarding the outcome while penalizing the path drives violation-free episodes from near zero to ≈100%—a deployability constraint the outcome reward is blind to, so outcome-only GRPO never reaches it at any budget—at no cost to task success (§3; on TerminalBench the same mechanism commits ≈6× fewer harmful actions at *equal* success). *(b) Reward progress where reachable.* Where the environment can verify a step of progress, a dense potential reaches competence in fewer costly rollouts than outcome-only training (§4). Mean over seeds; bands ±1 s.d.

1 Introduction

Consider an agent that acts in the world on a person's behalf: it calls a bank, works through the voice menu, waits on hold, and speaks with a representative to resolve a billing dispute. Each step is a real, often irreversible interaction—a live call, not a simulator step that can be reset and replayed a million times. A growing share of deployed agents work in exactly this regime [Pine AI, 2026a], and it differs from the math and code benchmarks that trained today's models in two ways this paper is about.

First, deployability depends on the *path*, not the outcome alone. A good outcome is necessary but not sufficient: an agent that resolves the task by calling a user who has declined to answer, acting outside business hours, or skipping a bank's required authentication is not deployable, however good the result. The outcome reward cannot express these constraints—doing more of the forbidden thing often *raises* the resolution rate—because they live on the path, not in the outcome. A deployable agent needs both, and an outcome-only one violates the path exactly when violating it helps.

Second, the agent learns *online*, from its own costly live interactions, and must learn from *few* of them. Math and code are the opposite regime: a cheap simulator lets a policy attempt a problem millions of times, so sample efficiency hardly matters—if learning is slow, run more rollouts. A real call cannot be replayed. An agent that learns from one representative that a bank requires a date of birth and the last four account digits must carry that to the next user, not rediscover it by trial and error; here sample efficiency is the difference between a system that improves online and one that cannot.

Reinforcement learning from verifiable rewards (RLVR) [DeepSeek-AI, 2025, Lambert et al., 2024] is blind to both needs. It trains on the one signal an environment checks cheaply—the outcome—to which the path is invisible and against which a harmful shortcut can score higher; and it wastes its most expensive interactions on the all-fail groups where a group-relative advantage is zero and the update does nothing. The instinct to go denser is to reward *progress* [Lightman et al., 2024, Wang et al., 2024, Setlur et al., 2025, Yu et al., 2024, Feng et al., 2025]. But progress is the hard-to-verify direction: judging whether an agent is on the right track is the whole problem it is solving, so one trains a critic the policy learns to fool [Cheng et al., 2025, Kazemnejad et al., 2024, Gao et al., 2023] or hand-crafts a proxy.

We take the asymmetry seriously. Agentic environments are *asymmetric verifiers*: they cheaply flag a *bad* move—a call before its precondition, an action out of hours—but cannot certify progress. The dense signal they can supply is a *penalty on the path*, not a reward on progress, and the folklore against penalties is a wiring error. A penalty *alone* fails—its cheapest maximizer is to stop acting, collapsing into a useless *inaction trap*. But a penalty is not meant to drive the task; the outcome reward does. Wired *alongside* it—reward the outcome, penalize the path—the penalty teaches the outcome-neutral constraints deployment requires, from the verifiable side of the ledger.

The same asymmetry has a second edge. Where the environment *can* verify a step of progress—a precondition discharged, a subgoal reached—that progress can be paid as a dense *potential*, densifying the sparse outcome on exactly the all-fail groups that were wasting rollouts. A single quantity governs both. A group-relative advantage *is* a within-group variance; it is zero on the all-failing groups early in training and the all-succeeding groups late, so the outcome is blind at both ends. A dense signal helps only where it supplies variance the outcome lacks, and only where the policy reaches the states that carry it: a verifiable penalty

satisfies this by construction, a potential only where partial progress is reachable. The penalty is the always-available half, the potential the reachability-gated half. We validate both on tractable proxies for the deployment setting (system-administration and customer-service tasks, a shell benchmark, a theorem prover, software repair); the transfer we claim is of the mechanism, not of numbers to live traffic.

Contributions.

- **A within-group-variance account** of when a dense process signal can help group-relative agentic RL: it must supply reachable within-group variance the outcome lacks (§2). The account is symmetric—outcome-only RL is blind on all-fail *and* all-success groups—and it separates the two verifiable path signals a deployed agent needs: penalties (reliably reachable) and progress potentials (reachable only sometimes).
- **Penalize the path, reward the outcome**: a two-channel recipe in which a per-action verifiable penalty robustly teaches the outcome-neutral constraints that make an agent deployable. Across diverse tasks and model scales it attains the task with near-zero violations where outcome-only training succeeds but violates every episode, and on a real agentic benchmark it cuts harmful actions sixfold at equal success (§3).
- **Four design rules for verifiable penalties**, each validated by ablation, including the inaction trap as the reason a penalty must accompany—never replace—the outcome reward (§3.1).
- **Sample efficiency from verifiable progress**: the *same* channel's credit, paid for verifiable progress rather than compliance, becomes a dense *potential* that addresses the second deployment need—learning from few costly rollouts. It turns the dead all-fail updates that dominate early training into learning where progress is reachable, and is vacuous where it is not (§4).

2 The Verifiable Path Channel

Group-relative RL. Modern agentic RL largely dispenses with a learned value function and estimates advantages by comparison *within a group* of rollouts of the same task [Shao et al., 2024, DeepSeek-AI, 2025]. For a prompt, one samples a group of G trajectories, scores each with a reward, and sets each trajectory's advantage to its reward minus the group mean (often divided by the group standard deviation). There is no critic: the other members of the group *are* the baseline. This is what makes verifiable rewards so effective—a rule or a test suite scores a trajectory directly—and it is the setting for everything that follows.

The advantage is a within-group variance. Because the baseline is the group mean, a trajectory contributes gradient only to the extent its reward *differs* from its group-mates. If every rollout in a group receives the same reward, all advantages are zero and the group produces no update. Under a binary outcome reward this happens in two regimes (Figure 2): the *all-fail* groups that dominate early training on long-horizon tasks, and the *all-succeed* groups that dominate once a task is nearly solved. Group-relative RL is thus blind at both extremes of the success rate—a fact the field half-acknowledges by *discarding* such groups (dynamic sampling in DAPO drops prompts that are all-correct or all-wrong [Yu et al., 2025]) rather than filling them with signal.

When a dense signal can help. A dense process signal is useful exactly when it restores variance the outcome has lost. Write the shaped reward of a trajectory as the outcome plus β times a process term, $R = O + \beta\,\Phi$. The group's reward variance—the quantity that becomes gradient—then decomposes as

$$\mathrm{Var}_G(R) \;=\; \mathrm{Var}_G(O) \;+\; \beta^2\,\mathrm{Var}_G(\Phi) \;+\; 2\beta\,\mathrm{Cov}_G(O,\Phi). \tag{1}$$

On an all-fail (or all-success) group the first term is zero, so *all* usable gradient must come from the process term's own within-group variance $\mathrm{Var}_G(\Phi)$. This gives a single, checkable condition: a process signal helps only where its within-group variance is nonzero—and that variance is realized only if the policy actually *reaches* differing intermediate states across the group.

Both conditions—finer than the outcome, and reachable—are met by *one* per-action channel, used two ways. The channel attaches a verifiable signal to the action that triggers it: a penalty $-\lambda$ for a verified *bad* move (a call before its precondition), a credit $+\beta$ for a verified *good* one (the precondition discharged, a subgoal reached). Used as a *penalty*, it almost always carries within-group variance—early in training some rollouts take the bad action and some do not, both cheap to sample and check—so it is the *always-reachable* use. Used as a *potential*—the same $+\beta$ credit paid for verifiable progress—it carries variance only where the policy reaches differing partial progress, which on hard tasks it usually does not (Figure 2, right), so it is the *reachability-gated* use. The rest of the paper is these two uses of one channel: penalize the path for deployability (§3), and reward verified progress for sample efficiency (§4).

3 Penalizing the Path: Verifiable Constraints for Deployable Agents

Two channels. We keep the outcome reward and add a second, per-action channel (Figure 3). At each step, a deterministic rule engine—a pure predicate over the state before the step and the action taken—checks the action. A *violation* (a destructive command, a call issued before its required precondition) attaches a penalty $-\lambda$ to that action's tokens; a *discharge* of a pending obligation (performing the precondition) attaches a credit $+\beta$. The two channels are normalized separately, so the sparse path signal is not diluted or cancelled by the outcome channel, then combined. "Penalize the path" is therefore literal: the gradient lowers the probability of the specific offending action, in the specific context where it occurred, while the outcome channel scales whole trajectories by success. We call such a signal a *verifiable penalty*: a deterministic predicate over the pre-action state and the action, checkable the moment it fires and independent of the eventual outcome. Crucially, the rules we penalize are *outcome-neutral*—following or breaking them does not change whether the task is solved—so this is signal the outcome reward can never provide. The full recipe pairs the penalty with its discharge, seeds a handful of scripted-compliant demonstrations so the compliant action is reachable, and anneals the shaping off once compliance saturates; §3.1 shows each of these earns its place.

The recipe attains the task with near-zero violations. We test on a suite of diverse system-administration and customer-service tasks—controllable proxies for the deployment setting—whose outcome-neutral rules are exactly the operating constraints of §1 in miniature: *verify a precondition before you act*, *do not repeat a request that was just refused*, *read before you mutate*. Drawn from a large pool and evaluated on held-out instances, outcome-only training learns to *solve*

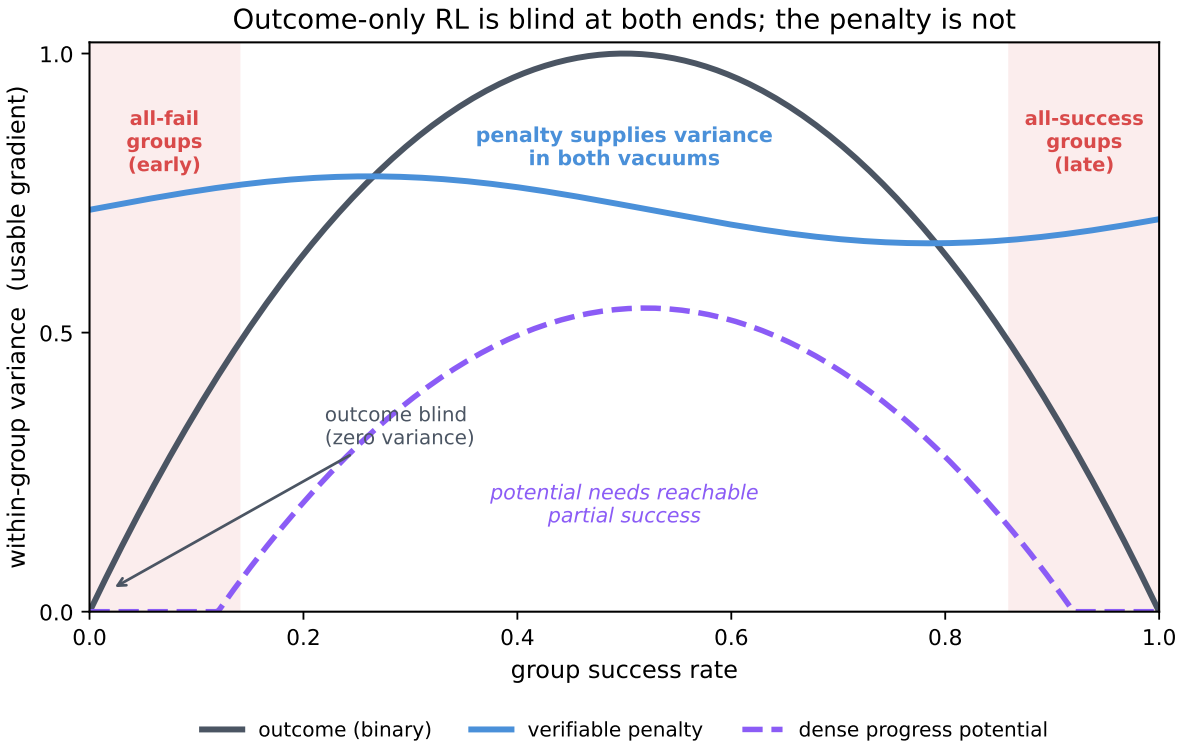

Figure 2. Outcome-only RL is blind at both ends of the success rate. A group-relative advantage is a within-group variance, which the binary outcome drives to zero on all-fail groups (early training) and all-succeed groups (late training). A dense process signal helps only by supplying within-group variance in these vacuums—and only where the policy reaches the intermediate states that carry it. A verifiable penalty (bad moves are easy to sample) meets this reliably; a dense progress potential (partial success is rare on hard tasks) does not.

the tasks but violates these rules on essentially every episode: the constraints are invisible to its reward, exactly as "did not call outside hours" is invisible to whether the dispute was resolved. Adding the penalty channel—reward the outcome, penalize the path—drives violations to near zero while keeping task success high, across five seeds at 4B and holding from 1.7B to 8B (Figure 4). The agent does not achieve this by going quiet: it takes as many actions as before, simply cleaner ones. This is the positive result in its cleanest form: a verifiable penalty teaches a real, deployable behavior—respecting the constraint itself—that the outcome reward cannot.

It transfers to a real agentic benchmark. The synthetic domains let us control the rules; to test the same mechanism in the wild we train on TerminalBench, where each turn is a real shell command in a task's container and the environment flags destructive actions—the `rm -rf` and drop-the-database shortcuts an outcome reward would happily take, now checked against a live filesystem. Task success there is at the floor for a small model, which makes it a clean test of the harm axis alone: at *equal* task success, the harm penalty cuts destructive actions *sixfold* relative to outcome-only training, while the policy issues *more* productive commands, not fewer (Table 1, five seeds). The reduction is large relative to its variance—the per-seed distributions do not overlap—and it is not passivity. A verifiable penalty bounds the path independently of whether the task is solved, which is exactly the property a deployable agent

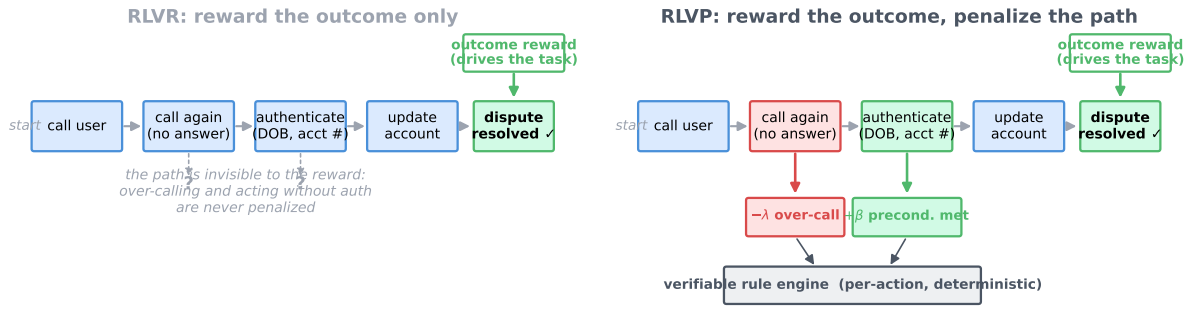

Figure 3. The verifiable path channel, in the real-world (phone-agent) setting. *Left (RLVR):* outcome-only training sees a single terminal reward and is blind to the path—over-calling and acting without authentication go unpenalized. *Right (RLVP):* a deterministic per-action rule engine attaches a verifiable signal to the responsible action—a penalty $-\lambda$ for a bad move (over-calling), a credit $+\beta$ for a good one (a precondition discharged)—alongside the same outcome reward. The penalty use (§3) bounds the path for deployability; the same $+\beta$ credit, paid for verifiable progress, is the potential of §4.

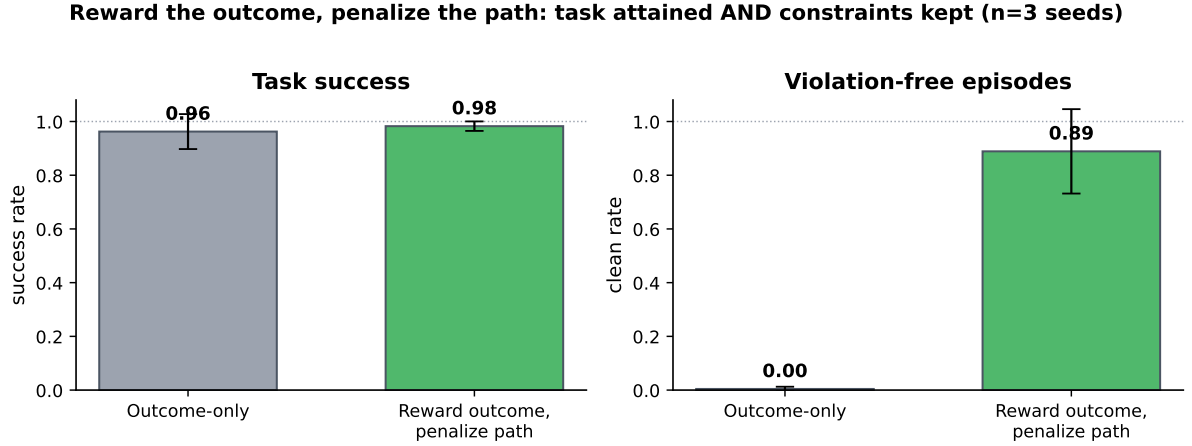

Figure 4. Reward-the-outcome-penalize-the-path attains the task *and* the constraints. Across diverse held-out tasks and model scales, outcome-only training solves the task but violates the outcome-neutral rules on nearly every episode (low "clean" rate). The two-channel recipe keeps task success high while driving violation-free episodes to near-100%, with small seed variance. Numbers are means over five seeds; error bars are one standard deviation.

needs.

3.1 Design rules for the penalty

A penalty is powerful but easy to misuse. The variance account and our ablations (Figure 6) yield four rules that, together, are the difference between the robust behavior above and a policy that stops working (Figure 5).

1. Penalize verifiable *actions*, not lack of progress. A good penalty targets a specific machine-checkable bad action—*a* destructive command, *a* call without its precondition. It must not target the *absence* of progress ("penalize an unproductive step"), because the cheapest way to make no unproductive steps is to make no steps. Penalize commission, never omission.

Table 1. Harm reduction at equal outcome (TerminalBench, Qwen3-4B, 5 seeds, mean ± std). Task success is statistically equal and at the floor (4B rarely solves the benchmark; the small RLVP–outcome gap is within one standard deviation), yet the un-gameable harm penalty cuts harmful actions roughly *sixfold*. The policy is not going passive: it takes about three times *more* productive actions while violating less. Harm lives on an axis the terminal outcome cannot see.

	RLVP (harm penalty)	**Outcome-only**
Task success	0.097 ± 0.060	0.122 ± 0.076 (equal within 1σ, at floor)
Violations / episode	$\mathbf{0.66 \pm 0.63}$	3.71 ± 0.52
Productive actions / episode	$\sim$**13**	$\sim$4

2. Keep the outcome reward as the task driver; never optimize a penalty alone. A penalty cannot be the primary learning signal. In the all-fail regime it is often the *only* signal with any slope, and its cheapest maximizer is to stop acting: the policy drifts into a penalty-free, useless optimum—the *inaction trap*. A pre-registered sweep makes this concrete: a pure penalty with no accompanying outcome reward collapses to zero task success on *every* seed, while every penalty-free signal survives (Appendix B.1, Figure 18). The outcome reward is what forces the agent to keep acting; the penalty only shapes *how*. Reward the outcome, and the penalty becomes a passenger that steers rather than an engine that stalls.

3. Pair the penalty with a discharge. A penalty says only what *not* to do. Adding a discharge—a $+\beta$ credit for performing the compliant action—gives a positive pull *toward* doing it right, so the policy escapes the neighborhood of the bad action instead of merely avoiding it. Removing the discharge measurably slows and destabilizes the acquisition of compliant behavior (Figure 6).

4. Make the compliant behavior reachable and its target un-gameable. A discharge can only reward behavior the policy actually samples; if the compliant path is never explored, no amount of shaping fires. Seeding training with a few scripted-compliant demonstrations supplies that reachability, after which the shaping can be annealed off once compliance saturates. And the penalized (and discharged) target must be the real action, not a learnable or hand-wavy proxy: a *learned* judge of "compliance" simply relocates the hack into the judge, which the policy then games (Appendix B).

Together these rules describe a narrow but reliable regime. The penalty is verifiable and outcome-neutral; it accompanies rather than replaces the outcome reward; it is paired with a discharge; and its target is reachable and un-gameable. The ablation in Figure 6 walks a policy from outcome-only, through the penalty channel stripped of its discharge and reachability seed, to the full recipe, and compliant-and-successful behavior appears robustly only when all four rules hold; the pure-penalty inaction trap itself—a penalty with no outcome reward—is isolated in the sweep of Appendix B.1.

4 Reward Verified Progress: Sample Efficiency Where Reachable

The path channel's credit has a second use. In §3 the $+\beta$ credit rewarded a compliant action, paired with a penalty; here we pay the *same* credit for verifiable *progress*—a falling goal count in

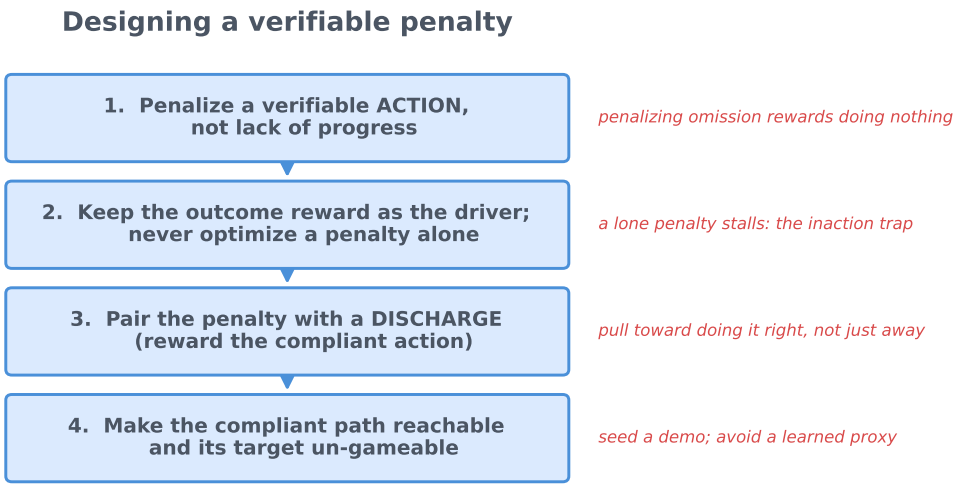

Figure 5. A design procedure for verifiable penalties. Before adding a penalty, a practitioner can check each rule in turn. Is there a verifiable *action* to penalize (not an absence of progress)? Is the outcome reward still the task driver? Is the penalty paired with a discharge for the compliant action? Is that compliant action reachable, and its target un-gameable? Only when all four hold does the penalty bound the path without stalling the task.

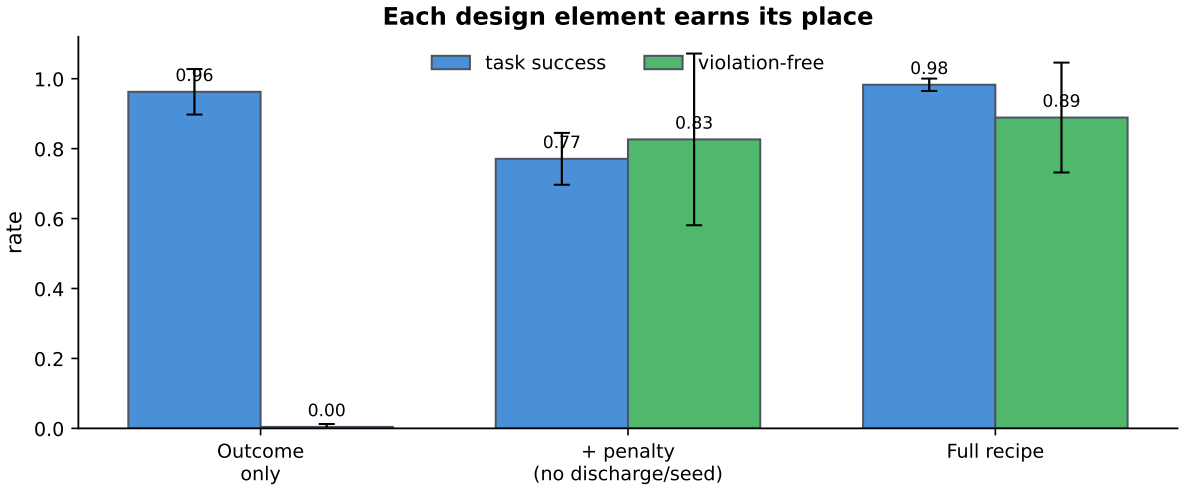

Figure 6. Each design element earns its place. On the diverse held-out tasks, outcome-only training solves the task but leaves violations high (low clean rate). Adding the verifiable penalty channel *without* its discharge and reachability seed does not reliably reach clean, successful behavior across seeds; the full recipe (penalty + discharge + seeded reachability + anneal) attains high success *and* near-zero violations with small seed variance. The *pure*-penalty inaction trap—a penalty with no outcome reward, which collapses to zero success on every seed—is isolated separately in the un-gameability sweep (Appendix B.1, Figure 18). Five seeds; bars show mean, whiskers one standard deviation.

a proof, a rising fraction of tests passed, a precondition reached—so it becomes a dense *potential* and serves the second deployment need, learning from few costly rollouts. It densifies the sparse outcome exactly on the all-fail groups that otherwise waste expensive interactions, so a policy reaches competence in fewer of them. We study it where partial progress is well-defined and cheap to iterate on: theorem proving and software repair, controllable proxies that let us run the many seeds an online phone-call setting never could. The variance account applies

Table 2. The aligned-potential matrix on miniF2F algebra (mean ± std over stable seeds; divergence-guard aborts counted separately). The aligned potential is the only arm simultaneously fast and reliable at both scales; the anneal ablation shows it needs no annealing; the AdamW rows are the outcome arm's stable-optimizer baseline.

scale	arm	iters to 0.9	AUC	final success	diverged
4B	aligned potential (Muon)	$\mathbf{4.4 \pm 0.5}$	$\mathbf{0.90 \pm 0.03}$	1.00 ± 0.00	**0/5**
4B	+ anneal	5.3 ± 0.5	0.85 ± 0.05	0.99 ± 0.02	0/3
4B	outcome-only (Muon)	7.0 ± 0.7	0.87 ± 0.02	1.00 ± 0.00	1/5
30B	aligned potential (Muon)	$\mathbf{5.4 \pm 1.0}$	$\mathbf{0.90 \pm 0.02}$	1.00 ± 0.00	**0/5**
30B	outcome-only (Muon)	8.5 ± 0.5	0.84 ± 0.00	1.00 ± 0.00	3/5
30B	outcome-only (AdamW)	19.2 ± 1.9	0.63 ± 0.07	0.97 ± 0.05	0/5

unchanged: this credit helps only where the policy reaches differing intermediate values, and the single thing that decides whether it does is *reachability*. On software repair the natural potential is the fraction of hidden tests an episode makes pass; across many rollouts of a capable model, *every* episode makes zero tests pass, so the potential's within-group variance is identically zero and it centers out to nothing (Figure 8, left). Reachability can be measured before training from a handful of base-policy rollouts, and the benefit of a dense potential tracks it. Reachability, not fragility, is the gate.

Where the potential *is* reachable, it robustly supplies the gradient the outcome lacks: it removes the dead all-fail updates that dominate early outcome-only training (zero vs. ∼16 of 40 iterations, every seed; Figure 8, right), and, with a bounded optimizer, converts that gradient into faster and more reliable learning. We quantify this on real theorem proving with a matched five-seed matrix at two scales (Figure 7, Table 2). The gain is a modest, seed-consistent speed-up under a matched optimizer and a large reliability advantage under an aggressive one: at 30B, outcome-only must choose between diverging under the fast optimizer and training several times slower under its stable one, while the aligned potential is fast *and* stable on every seed at both scales. These are not the dramatic single-run accelerations sometimes reported—our own single-run pilots produced those, and re-seeding erased them. Protocol and the divergence taxonomy are in Appendix A.

An aligned-potential recipe. Making a dense potential help *robustly* takes more than adding it; three ingredients matter. **(i) Alignment:** the potential must be *outcome-instrumental*—a quantity that moves as the task is solved (a falling goal count, a rising fraction of tests passed), carried as a token-attached discharge with its own normalizer. A generic *structural* proxy (read-before-write hygiene, tactic well-formedness) is misaligned and, once the task is mastered, over-optimized into a degraded ceiling. **(ii) Reachability:** its intermediate values must actually be sampled, measurable *before* training from $\mathrm{Var}_G(\Phi)$ on base-policy rollouts. **(iii) A bounded optimizer:** an aggressive update rule drives the shaped policy into entropy collapse or a gradient blow-up; a bounded rule (we use Muon) keeps it stable. Annealing after saturation, usually treated as mandatory, is *unnecessary* here—an outcome-instrumental potential has no misaligned pressure to relax, and the anneal ablation (Table 2) changes nothing outside noise.

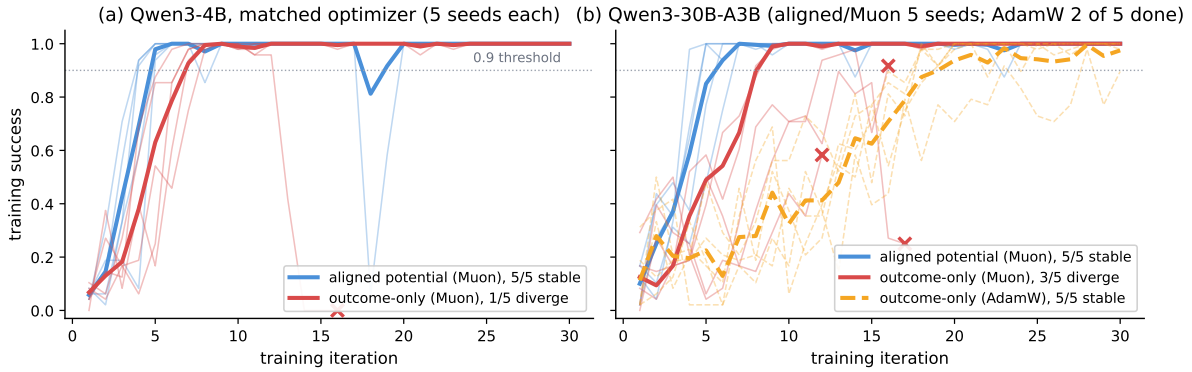

Figure 7. The aligned-potential recipe on real theorems (miniF2F algebra), five seeds per arm. Thin lines are seeds, thick lines the mean over stable seeds, × marks a divergence-guard abort. *(a)* Controlled same-optimizer comparison at 4B: the aligned potential crosses the 0.9 threshold in 4.4 ± 0.5 iterations vs. 7.0 ± 0.7 for outcome-only—faster on every seed—and never diverges, while outcome-only loses one seed to entropy collapse. *(b)* At 30B, outcome-only faces a speed–reliability dilemma: under Muon it diverges on three of five seeds; under AdamW it is stable on all five seeds but takes ∼3.6× longer to master the task (threshold 19.2 ± 1.9 vs. 5.4). The aligned potential is fast *and* stable on every seed at both scales.

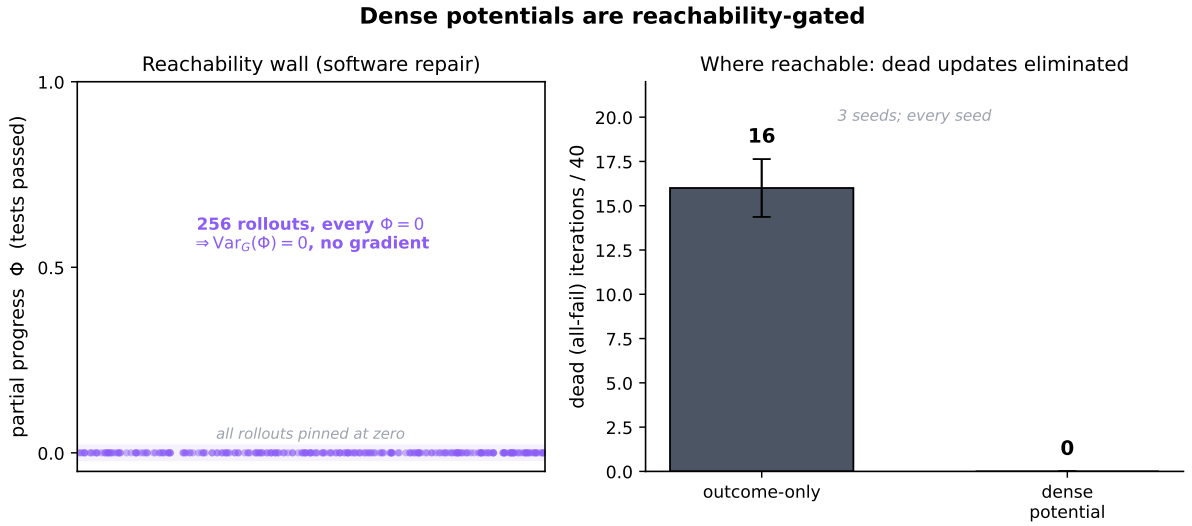

Figure 8. Dense potentials are reachability-gated. *Left:* on software repair the finer potential is unreachable—across many rollouts the policy makes zero partial progress, so its within-group variance is zero and it gives no gradient. *Right:* where the potential *is* reachable, it removes the dead all-fail updates that dominate early outcome-only training (zero versus ∼16 of 40 iterations, every seed), supplying gradient exactly where the outcome is blind. The gate is reachability, not fragility.

5 Discussion

Verifier asymmetry as a design principle. The practical takeaway is a reallocation. A verifier is a scarce, valuable thing, and the field has been spending it on the direction it is worst at—certifying progress—while avoiding the direction it is best at—catching bad moves. Spend it on penalties. When there is an outcome-neutral constraint a deployed agent must respect, and a verifiable bad action to attach it to, a penalty paired with a discharge and wired alongside

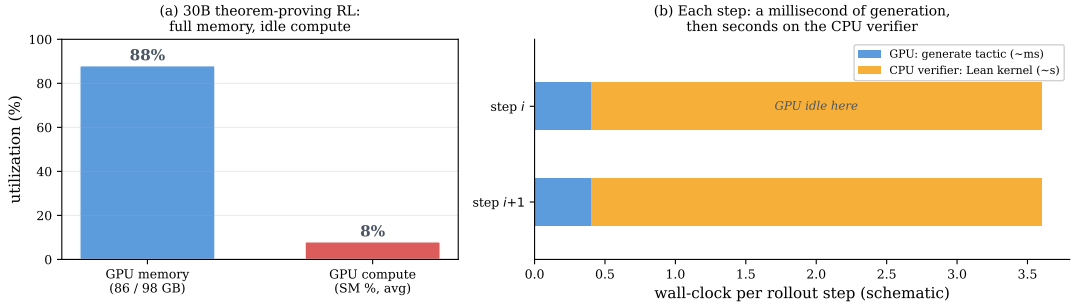

Figure 9. Verifiable agentic RL is often verifier-bound. In theorem-proving runs the GPU sits near-idle at full memory while the CPU proof kernel checks each step. The signal that makes a penalty cheap also moves the bottleneck off the accelerator.

the outcome reward will teach it, robustly and cheaply. When the only dense signal available is a progress proxy, first check that its intermediate values are reachable: where they are, it accelerates learning; where they are not, it supplies nothing and the outcome reward alone is the better choice.

A systems note. The property that makes a verifiable penalty cheap—a machine-checkable environment—tends to make the verifier a CPU process (a rule engine, a test runner, a container). In our theorem-proving runs the accelerator generated a step in milliseconds and then waited seconds on the proof kernel, leaving GPU utilization in the single digits at full memory (Figure 9). This is workload-dependent, not a law, but for the verifiable domains where penalties apply, throughput is gated by environment parallelism, and the right systems design co-locates many verifier workers per accelerator rather than scaling accelerators.

From proxies to deployment. The two properties we lean on—each rollout is expensive and irreversible, and the checkable signal is a path constraint rather than a progress judgment—are sharpest in the online real-world setting that motivates the work: an agent improving on live phone calls cannot afford wasted rollouts and must respect operating rules it is never rewarded for. That setting is also where controlled, many-seeded experiments are infeasible, which is why our evidence is on tractable proxies chosen to preserve that structure. The transfer we claim is of the *mechanism*—a verifiable penalty supplies variance the outcome lacks; a verifiable potential does so where progress is reachable—not of specific numbers to live traffic. Validating on a deployed online agent, where the sample-efficiency payoff is measured in real interactions saved, is the natural next step and the one we most want to take.

Limitations. Our cleanest real-task harm result is measured where task success is at the floor; the mechanism predicts it holds at higher success (a penalty's variance is reachable regardless of success), but demonstrating that on a capable model is the natural next step. Our penalties are hand-identified per domain from the environment's rules; automating their discovery from an environment's affordances is open. And the un-gameability sweep that anchors the inaction-trap result is high-variance at scale; we report its *survival pattern* rather than precise magnitudes.

6 Related Work

Group-relative RL and the zero-variance problem. Agentic RL has largely converged on GRPO [Shao et al., 2024] and the rule-based R1 line [DeepSeek-AI, 2025, Lambert et al., 2024], which replace the value critic with a within-group baseline. Under binary verifiable rewards, an output's advantage is then *definitionally* a within-group variance, and it collapses to zero whenever a group is uniformly solved or uniformly failed. The field has named this "advantage collapse" and responded in four ways: DAPO [Yu et al., 2025] *discards* zero-variance prompts via dynamic sampling, keeping only groups with $0 < \#\text{correct} < G$; Dr. GRPO [Liu et al., 2025] *removes* the difficulty-dependent standard-deviation normalizer; GSPO [Zheng et al., 2025] *stabilizes* the importance ratio at sequence level; and RL-ZVP [Le et al., 2025] *manufactures* a signal in zero-variance groups from output entropy. Notably, DAPO discards *all-correct* groups as well as all-wrong ones—an implicit acknowledgment that group-relative RL is blind at *both* extremes. We make the symmetry explicit and, rather than dropping the blind groups, ask which dense signal can *supply* the missing variance—finding that a verifiable penalty reliably can and a dense potential reliably cannot.

Process reward models and their gameability. Step-level supervision outperforms outcome supervision for verification [Lightman et al., 2024], with per-step labels obtainable automatically via Monte-Carlo rollouts [Wang et al., 2024]. Setlur et al. [2025] argue the useful process signal is a progress measure—a step-level advantage—closely paralleling our variance view, but instantiate it with a *learned* prover. A consistent body of evidence shows learned dense rewards are hacked under RL pressure: over-optimization follows clean Goodhart laws [Gao et al., 2023], capable agents exploit misspecification harder [Pan et al., 2022], summation-form PRM credit induces RL collapse [Cheng et al., 2025], and learned value credit is unreliable [Kazemnejad et al., 2024]. DeepSeek-R1 [DeepSeek-AI, 2025] abandoned PRMs because a model-based PRM "inevitably leads to reward hacking," while Yuan et al. [2024a] show an outcome reward model implicitly contains a PRM. We take these as motivation for a *verifiable, non-learned* penalty that cannot be gamed by construction, and as evidence for our fourth design rule.

Potential-based shaping: safe versus useful. Ng et al. [1999] prove potential-based shaping leaves the optimal policy unchanged for *any* potential, and Wiewiora [2003] show such shaping equals value initialization—it affects learning dynamics, not the fixed point. This certifies which dense signals are *safe*, not which are *useful*. Our account is orthogonal to the safety guarantee: among safe shapings, the useful ones supply within-group variance the outcome lacks, and only where reachable—which is why a penalty on always-sampled bad actions helps and a potential on rarely-reached progress does not.

Process rewards in agentic RL, and the fragility of RL gains. Step- and turn-level credit for agents has been explored via hierarchical critics [Zhou et al., 2024], learned step rewards [Yu et al., 2024, Wang et al., 2025a], and turn-level advantage estimators [Wei et al., 2025a]; the closest to a verifiable signal are GiGPO [Feng et al., 2025] and per-tool-call correctness [Qian et al., 2025]. Verifiable *outcome* rewards underpin agentic benchmarks and training [Yao et al., 2024, Barres et al., 2025, Pan et al., 2025, Wei et al., 2025b]. Almost all of this work assumes a cheap, resettable simulator, where sample efficiency is secondary—if learning is

slow, one runs more rollouts. The online real-world setting that motivates us inverts that assumption: each rollout is a costly, irreversible interaction, so turning the wasted all-fail rollouts into learning (via a reachable potential) and bounding the path without extra sampling (via a penalty) become first-order concerns rather than refinements. Wang and Ammanabrolu [2025] find dense turn-level rewards *accelerate* training but their effect on final performance depends on the policy-gradient estimator. This aligns with a broader reproducibility literature: pass@1 varies by up to ∼15% across seeds [Hochlehnert et al., 2025], random rewards can "improve" some models through optimization bias [Shao et al., 2025], one example suffices for much of the gain [Wang et al., 2025b], and at large k base models match RLVR models [Yue et al., 2025]. Our potential result sharpens this picture: a dense potential's benefit is gated by whether its intermediate values are reachable and by the use of a bounded optimizer, and the non-replications reported for such gains track those two conditions rather than an intrinsic fragility of the signal. Learned-judge rewards [Yuan et al., 2024b, Lee et al., 2023, Bai et al., 2022, Zheng et al., 2023] and off-policy demonstration mixing [Yan et al., 2025] are complementary; our fourth design rule uses the latter for reachability and warns against the former for un-gameability.

7 Conclusion

RLVR taught language agents from the one bit an environment checks cheaply—the final outcome. The instinct to go denser has been to reward the process, but the environment cannot verify progress; it can only verify mistakes. The dense signal it actually affords is a penalty on the path, not a reward on progress. Wired correctly—alongside the outcome reward, paired with a discharge, aimed at a reachable and un-gameable target—a verifiable penalty teaches the outcome-neutral constraints that make an agent deployable, robustly and across scales, while a lone penalty stalls and a dense potential helps precisely where its progress is reachable. The contribution is not a new optimizer but a reallocation of a scarce resource: spend the verifier on penalties that bound the path, and—where it can certify progress—on potentials that turn wasted rollouts into learning. For an agent that learns on live interactions, where every rollout is costly and every misstep visible, that reallocation is the difference between a system that can be deployed and improved online and one that cannot.

Acknowledgements

We thank Dingkun Hong, Dongjiang Li, and Wei Zhou of ByteDance for discussions that inspired the application of this method to AI coding. This paper was produced using Pine Copilot's voice-directed *whisper coding* workflow [Pine AI, 2026b], in which the authors specify, discuss, and review the work by voice while a coding agent—Claude Code with Claude Opus 4.8—carries out the planning, coding, experiments, and paper writing. We thank BSQL Networking for hosting the NVIDIA RTX PRO 6000 GPU.

References

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.

Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2-bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*, 2025.

Jie Cheng et al. Stop summation: Min-form credit assignment is all process reward model needs for reasoning. *arXiv preprint arXiv:2504.15275*, 2025.

DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Lang Feng et al. Group-in-group policy optimization for llm agent training. *arXiv preprint arXiv:2505.10978*, 2025.

Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *International Conference on Machine Learning (ICML)*, 2023.

Andreas Hochlehnert et al. A sober look at progress in language model reasoning: Pitfalls and paths to reproducibility. *arXiv preprint arXiv:2504.07086*, 2025.

Dingkun Hong. AI Coding: Practice and exploration, 2026. Keynote, ByteDance / Volcano Engine Force Conference, Beijing, June 2026. https://mp.weixin.qq.com/s/tJAinVKzZAqZrhiEvGU2Dg (accessed 2026-06-25).

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

Amirhossein Kazemnejad et al. VinePPO: Unlocking RL potential for LLM reasoning through refined credit assignment. *arXiv preprint arXiv:2410.01679*, 2024.

Nathan Lambert, Jacob Morrison, Valentina Pyatkin, et al. Tülu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024.

Thanh-Long V. Le, Myeongho Jeon, Kim Vu, Viet Lai, and Eunho Yang. No prompt left behind: Exploiting zero-variance prompts in LLM reinforcement learning via entropy-guided advantage shaping. *arXiv preprint arXiv:2509.21880*, 2025.

Harrison Lee, Samrat Phatale, Hassan Mansoor, Kellie Lu, Thomas Mesnard, Colton Bishop, Victor Carbune, and Abhinav Rastogi. RLAIF: Scaling reinforcement learning from human feedback with ai feedback. *arXiv preprint arXiv:2309.00267*, 2023.

Hunter Lightman, Vineet Kosaraju, Yura Burda, et al. Let's verify step by step. *International Conference on Learning Representations (ICLR)*, 2024.

Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding r1-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.

Andrew Y Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *International Conference on Machine Learning (ICML)*, pages 278–287, 1999.

Alexander Pan, Kush Bhatia, and Jacob Steinhardt. The effects of reward misspecification: Mapping and mitigating misaligned models. In *International Conference on Learning Representations (ICLR)*, 2022.

Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. SWE-Gym: Training software engineering agents and verifiers. In *International Conference on Machine Learning (ICML)*, 2025.

Pine AI. Pine AI. https://www.19pine.ai, 2026a. An AI assistant that completes real-world tasks—phone calls, customer service, account and subscription management—on a user's behalf. Accessed 2026-07.

Pine AI. Pine AI: The most natural human-computer interface is your voice. Blog post, 2026b. URL https://www.19pine.ai/blog/pine-ai-the-most-natural-human-computer-interface-is-your-voice. Accessed 2026-06-28.

Cheng Qian et al. Toolrl: Reward is all tool learning needs. *arXiv preprint arXiv:2504.13958*, 2025.

Amrith Setlur, Chirag Nagpal, et al. Rewarding progress: Scaling automated process verifiers for LLM reasoning. In *International Conference on Learning Representations (ICLR)*, 2025.

Rulin Shao et al. Spurious rewards: Rethinking training signals in RLVR. *arXiv preprint arXiv:2506.10947*, 2025.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Hanlin Wang, Chak Tou Leong, Jiashuo Wang, Jian Wang, and Wenjie Li. SPA-RL: Reinforcing LLM agents via stepwise progress attribution. *arXiv preprint arXiv:2505.20732*, 2025a.

Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. Math-shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

Ruiyi Wang and Prithviraj Ammanabrolu. A practitioner's guide to multi-turn agentic reinforcement learning. *arXiv preprint arXiv:2510.01132*, 2025.

Yiping Wang et al. Reinforcement learning for reasoning in large language models with one training example. *arXiv preprint arXiv:2504.20571*, 2025b.

Quan Wei, Siliang Zeng, Chenliang Li, William Brown, and Mingyi Hong. Reinforcing multi-turn reasoning in LLM agents via turn-level reward design. *arXiv preprint arXiv:2505.11821*, 2025a.

Yuxiang Wei, Olivier Duchenne, Jade Copet, others, and Sida I. Wang. SWE-RL: Advancing LLM reasoning via reinforcement learning on open software evolution. *arXiv preprint arXiv:2502.18449*, 2025b.

Eric Wiewiora. Potential-based shaping and Q-value initialization are equivalent. *Journal of Artificial Intelligence Research (JAIR)*, 19:205–208, 2003.

Jianhao Yan et al. Learning to reason under off-policy guidance. *arXiv preprint arXiv:2504.14945*, 2025.

John Yang, Kilian Lieret Zhang, et al. SWE-smith: Scaling data for software engineering agents. *arXiv preprint arXiv:2504.21798*, 2025.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

Yuanqing Yu et al. Steptool: Enhancing multi-step tool usage in llms through step-grained reinforcement learning. *arXiv preprint arXiv:2410.07745*, 2024.

Lifan Yuan, Wendi Li, Huayu Chen, Ganqu Cui, Ning Ding, Kaiyan Zhang, Bowen Zhou, Zhiyuan Liu, and Hao Peng. Free process rewards without process labels. *arXiv preprint arXiv:2412.01981*, 2024a.

Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models. *arXiv preprint arXiv:2401.10020*, 2024b.

Yang Yue et al. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? *arXiv preprint arXiv:2504.13837*, 2025.

Chujie Zheng, Shixuan Liu, et al. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.

Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. minif2f: a cross-system benchmark for formal olympiad-level mathematics. *International Conference on Learning Representations (ICLR)*, 2022.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *arXiv preprint arXiv:2306.05685*, 2023.

Yifei Zhou, Andrea Zanette, Jiayi Pan, Sergey Levine, and Aviral Kumar. ArCHer: Training language model agents via hierarchical multi-turn RL. In *International Conference on Machine Learning (ICML)*, 2024.

A Full Experiments on Dense Potentials

The main text treats the dense progress *potential* as the reachability-gated half of the process signal (§4). This appendix gives the full experiments. The picture is consistent with the variance account: where a potential's intermediate values are *reachable* it supplies gradient the outcome lacks and accelerates training, and where they are unreachable it is exactly vacuous. Converting the extra gradient into a stable success gain requires a bounded optimizer—an

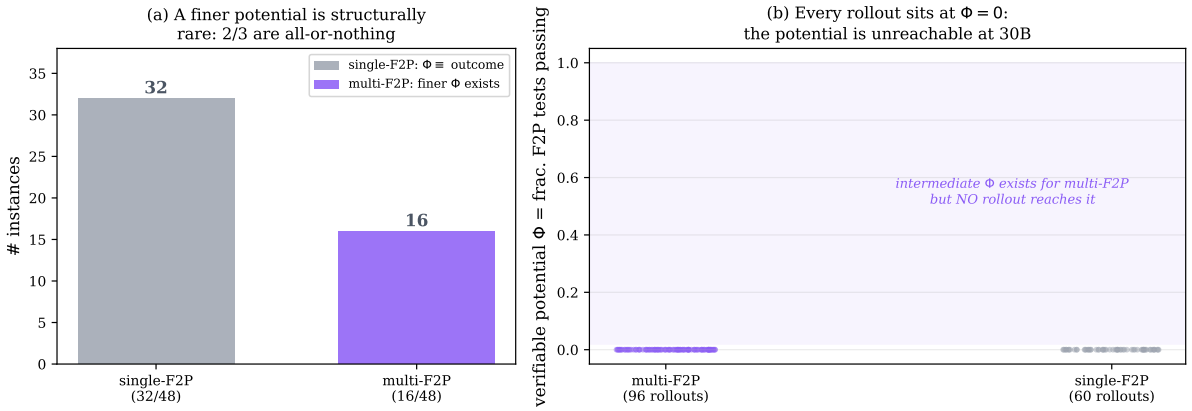

Figure 10. In software repair the finer potential is structurally rare and empirically unreachable. *(a)* Of 48 single-file-fix SWE-bench instances, two-thirds have a single failing test, so the test-fraction potential equals the binary outcome; only one-third admit a finer Φ. *(b)* For those, a 30B policy's rollouts never realize it: all 156 rollouts score $\Phi = 0$. The band of intermediate values exists but is empty—zero within-group variance, hence zero gradient.

aggressive update rule can destabilize the densely shaped policy, an optimization caveat we return to in §A.3. Unless noted, we train Qwen3 [Team, 2025] with our own GRPO, place any dense credit on the responsible tokens with a separate per-channel normalizer, and count efficiency in *episodes generated* so that resampling costs are visible.

A.1 Reachability is measurable before training, and it gates the benefit

The condition the variance account makes operative is reachability: a potential Φ helps only where the policy realizes differing intermediate values across a group, i.e. $\mathrm{Var}_G(\Phi) > 0$. This can be checked *before* spending training compute, from a handful of base-policy rollouts.

On SWE-bench software repair [Jimenez et al., 2024] the natural potential is the fraction of hidden `FAIL_TO_PASS` tests an episode makes pass. Two facts make it vacuous (Figure 10). Structurally the finer potential barely exists: two-thirds of instances have a single failing test, so Φ collapses to the binary outcome, and only a third admit any partial progress. Empirically even that third is unreachable: across 156 rollouts of a 30B policy *every* episode scores $\Phi = 0$, so $\mathrm{Var}_G(\Phi)$ is identically zero and a Φ-based reward centers out to no gradient. The cause is not gameability but unreachability—there is no realized intermediate state for any reward to act on.

Mapping the same no-training proxy $\mathrm{Var}_G(\Phi)$ across model capability (0.6–4B) and outcome sparsity (Figure 11a) reproduces the structure the account predicts: it rises with capability and peaks in the sparse-but-reachable corner. The measured dense-reward benefit tracks it (Figure 11b): ≈ 0 where $\mathrm{Var}_G(\Phi) \approx 0$ (the SWE null; the too-weak small models) and large where it is high (the 4B reachable chain, dense 0.34 vs. outcome 0.01; real theorem proving, below). Benefit appears only past a reachability threshold. We could not fill the map's interior with controlled *training* points—small-scale synthetic training is too learning-rate-sensitive, most cells degenerating to zero for both arms—so the benefit axis rests on the real-domain anchors plus one clean controlled point.

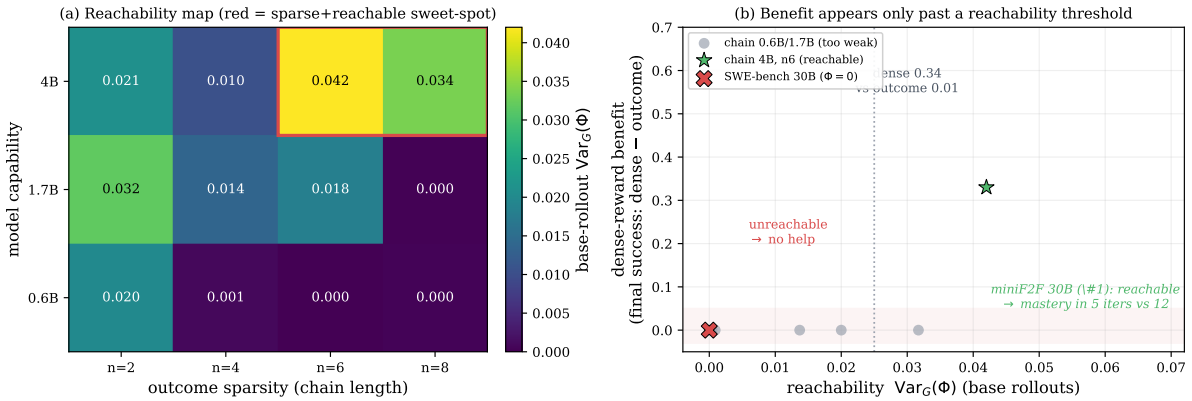

Figure 11. Reachability, measured cheaply, gates the dense-reward benefit. *(a)* $\mathrm{Var}_G(\Phi)$ on base-policy rollouts (no training) over model capability $\times$ outcome sparsity: it rises with capability and peaks in the sparse-but-reachable corner. *(b)* The benefit of a dense reward (final-success gap, dense $-$ outcome) against that same probe: zero where $\mathrm{Var}_G(\Phi)\approx 0$ and large where it is high. Benefit appears only past a reachability threshold.

A.2 Where the potential is reachable, it removes dead updates and accelerates

When the potential's intermediate values are reachable and point the same way as success, a dense signal is genuinely useful. We instantiate it on a family of N-*stage chained* file-operations tasks (tools `list_dir`, `read_file`, `write_file`, `delete`, `run_tests`, `submit`); success requires all N stages, so N tunes base difficulty (N=4 sits near 8% base success). The dense signal is a verifiable progress potential carried on a token-attached channel (a penalty when a call violates a rule—overwrite a file never read, submit an untested change—and a credit when a call discharges a pending obligation).

Dead updates. We count the fraction of iterations whose update is *dead*: an all-fail group with no reward spread (Figure 12). Outcome-only GRPO is dead on 65% of iterations. DAPO [Yu et al., 2025], built to dodge all-fail groups by resampling, reaches 54% but pays a 5.6$\times$ generation tax. The aligned potential cuts dead updates to 8% at no extra sampling, by producing gradient from the very failures the outcome reduces to zero. The cure is the credit scheme, not the budget—and it is contingent on reachability: on the silent-precondition gate (§A.4), where the pivotal action is never sampled, every reward-only method is dead almost always (outcome and DAPO 100%, the potential 76%), because a discharge can fire only on behavior the policy attempts.

Speed. On held-out success against episodes generated (Figures 13, 14) outcome-only crawls, spending its budget on dead updates, then converges only at the end; the aligned potential rises immediately from the early all-fail groups. Reimplemented dense baselines (GiGPO-style step advantages [Feng et al., 2025], StepTool-style per-call rewards [Yu et al., 2024]) also outpace outcome-only—any *admissible* dense signal helps. The gain also holds on real theorems (miniF2F [Zheng et al., 2022]) at 4B and 30B, but only in the multi-seed form quantified in §A.3: a single-run pilot of ours showed a dramatic 2.4$\times$ speed-up that *flipped sign* when the identical configuration was re-run (rollout sampling is unseeded in the serving engine, and MoE training at this scale is bimodal), so we report only the five-seed matrix (Figure 7, Table 2):

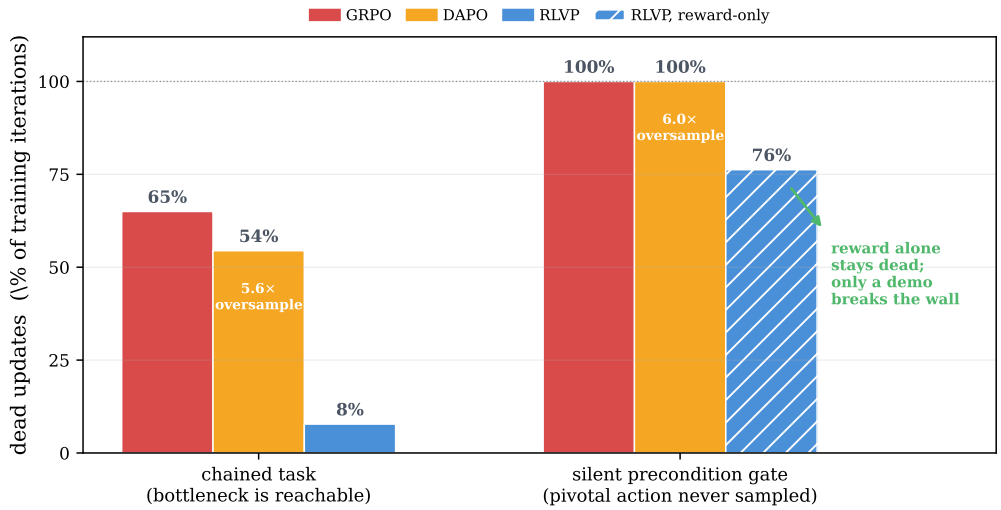

Figure 12. Dead updates across two regimes. Fraction of iterations with an all-fail, zero-spread group (Qwen3-4B, 3-seed mean). *Chained task* (bottleneck reachable): outcome-only 65%, DAPO 54% at a 5.6× generation tax, the aligned potential 8%. *Silent precondition gate* (§A.4, pivotal action never sampled): outcome and DAPO dead on *every* iteration, the reward-only potential 76%; only a demonstration seed breaks the wall. The cure is a *reachable, admissible* signal, not density.

a $\sim$1.6× speed-up to mastery under a matched optimizer, consistent on every seed, plus the reliability advantage. Both arms saturate—the claim is speed and reliability, not final success.

A.3 Converting the gradient to success: protocol, and the matched matrix

The extra gradient a reachable potential supplies must still be turned into task success, and here the decisive factor is the optimizer, not the potential. Early runs read this as "fragility," but the cause was two experimental artifacts. First, an aggressive, unbounded update rule can destabilize a densely shaped policy: at one fixed learning rate, a theorem-proving run anneals cleanly to full success (gradient norm < 1.5, entropy decaying to zero) while an otherwise-identical sibling suffers a gradient blow-up (norm $> 10^9$), its entropy explodes, and success collapses to zero and never recovers—the same signal, the same rate, decided by optimization stability. On the chained task the opposite failure appears: the policy collapses to a zero-entropy degenerate output, so every rollout is identical, the within-group variance is zero, and the update dies. A *bounded* optimizer (we use Muon, with a divergence guard that aborts on entropy collapse or gradient spikes) removes both failure modes. Second, per-iteration training success measured on a handful of freshly sampled tasks is too noisy to read a speed-up from—it swings between extremes iteration to iteration—so we evaluate on a fixed held-out task set. The result robust to all of this—the mechanism the paper relies on—is dead-update elimination: zero dead iterations against $\sim$16 of 40 for outcome-only, on *every* seed.

The matched matrix: five seeds, two scales. With the bounded-optimizer protocol fixed (aligned goal-progress potential as a token-attached discharge, Muon, a divergence guard, 30 iterations on miniF2F algebra), we ran the full matrix—aligned potential versus outcome-only, five seeds per arm, at 4B and 30B, plus a three-seed anneal ablation and a 30B AdamW

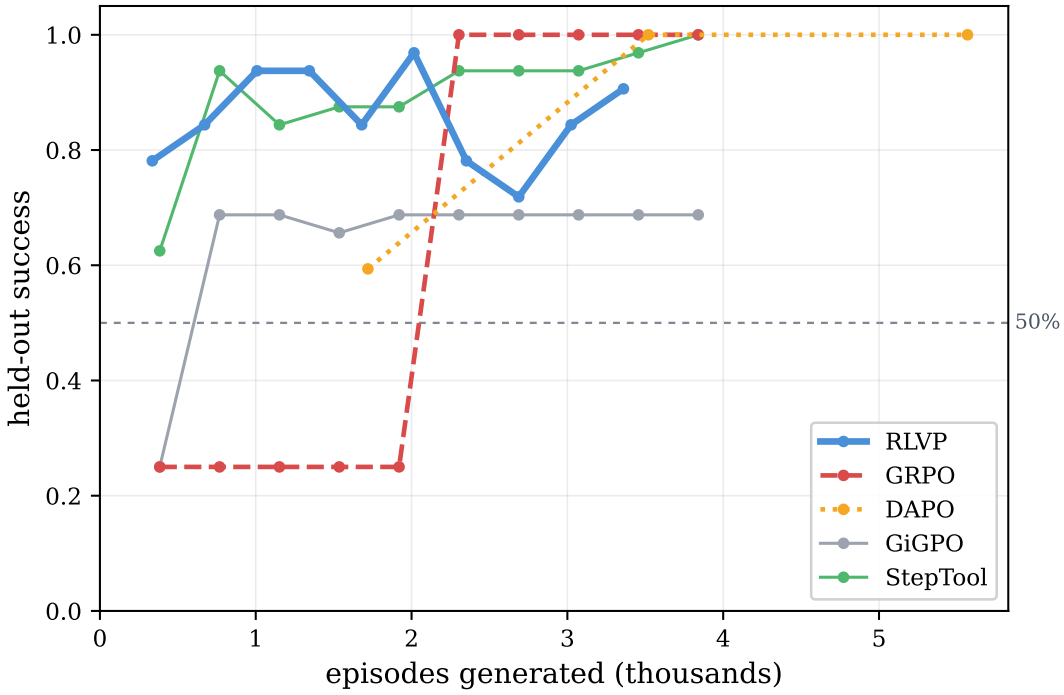

Figure 13. Sample efficiency on the calibrated hard task (N=4). Held-out success vs. episodes generated. Outcome-only sits near base rate for most of training; the aligned potential climbs immediately from the all-fail groups; dense baselines also outpace outcome-only. The x-axis counts episodes *generated*, including DAPO's resampling.

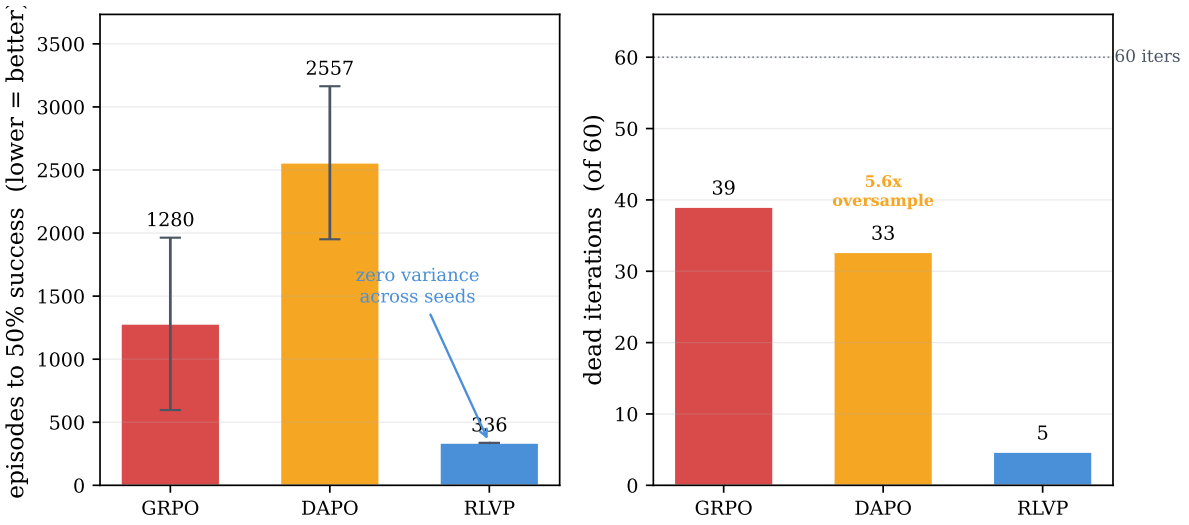

Figure 14. Efficiency over three seeds. *Left:* episodes to the success threshold (lower better)—the potential is several times faster than GRPO and DAPO. *Right:* the mechanism—GRPO and DAPO burn most iterations on dead or stalled updates, and DAPO generates 5.6× the episodes it uses. Note the large GRPO error bar: the all-fail lottery on the outcome side, which the dense potential's reachable within-group variance avoids.

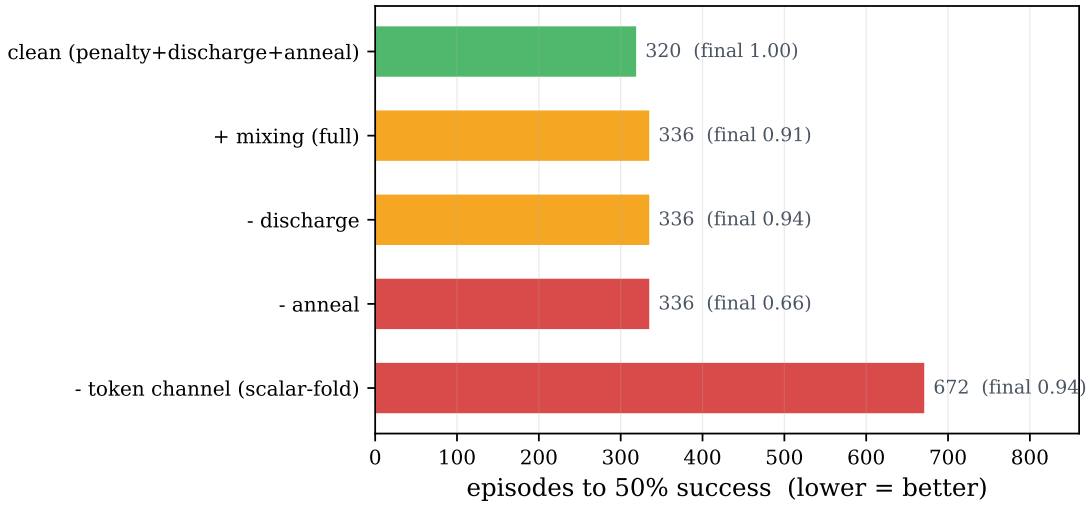

Figure 15. Component attribution for the potential on $N{=}4$ (episodes to threshold, converged success annotated). The token-attached credit channel is the efficiency lever (folding into a scalar return doubles the cost); annealing protects the ceiling; the clean recipe is best. A per-step length cost, useful only on very long horizons, hurts here.

outcome baseline; all magnitudes are in Table 2. Because rollout sampling in the serving engine is unseeded, "seeds" are i.i.d. draws of the whole run, so per-arm consistency is the meaningful statistic. Three facts are seed-robust. **(1) A modest, consistent speed-up** under a matched optimizer: the aligned potential reaches the threshold sooner and has higher area-under-curve on every seed-matched pair at 4B. **(2) Reliability:** it completes every run across both scales without a divergence abort, while outcome-only loses seeds to entropy collapse (4B) and gradient explosion (30B)—the dense, always-relevant gradient appears to *regularize* where the bursty sparse signal blows up under the same rate. **(3) The outcome arm's dilemma at 30B:** outcome-only is competitive under the fast optimizer only when it survives (it diverges on most seeds), and stable under its safe optimizer only at several times the potential's time-to-mastery. The aligned potential requires no such choice.

Anatomy of the gain. When the potential does win, ablating on $N{=}4$ (Figure 15) attributes the speed to the token-attached channel: folding the same rule rewards into the scalar return doubles the episodes to threshold. Annealing the shaping off after saturation protects the final ceiling (unrelaxed pressure drifts the policy toward over-compliant, bloated episodes—a mild inaction-trap tendency). The discharge credit is the positive signal that escapes all-fail groups; removing it leaves more dead iterations and a lower ceiling. Finally, replacing every hand-written rule with three universal meta-rules derived only from per-tool category tags (observe/mutate/verify/terminal) and the environment's error codes recovers the hand-engineered efficiency almost exactly: in well-structured environments the specification cost can be paid by metadata the tool schemas already expose.

A.4 Reachability governs the ceiling, not just the speed

Sample efficiency is speed to a ceiling the chained tasks eventually reach. To ask whether a process reward can raise the *ceiling*, we build a task that is hard to *discover* rather than hard to

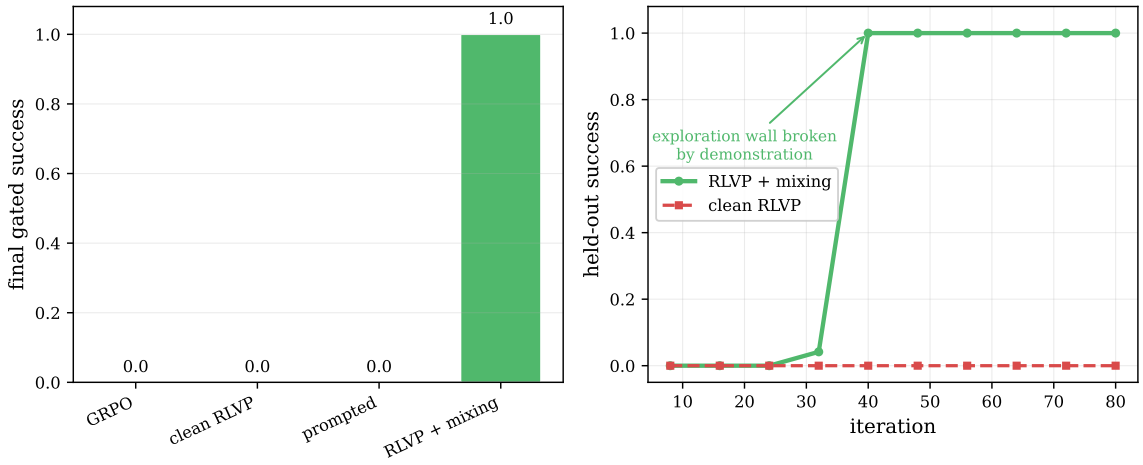

Figure 16. A non-saturating ceiling test, and the discovery wall. *Left:* on the silent-gate task every reward-only method (outcome, DAPO, clean potential, even rules-in-prompt) converges to zero—the precondition is never sampled, so the discharge never fires. *Right:* a single synthesized demonstration through the process channel breaks the wall to perfect held-out success, a sharp phase transition.

compute: a *silent precondition gate*, where to write a protected file the agent must first read an access-control file and request access; skip either and the write silently no-ops (reports success, does nothing). This is the bank-authentication problem of the introduction in controlled form—a required precondition an agent cannot stumble onto by trial and error and must be told—and it is where a purely reward-driven online learner would stall indefinitely. Calibration confirms the trap is total—the base model never reads the access file even when the rule is in its prompt. Then (Figure 16) *every* reward-only method (outcome, DAPO, the clean potential) fails to zero, because the discharge for reading the access file can only reward that behavior if the policy samples it, and it never does. A single rule-synthesized demonstration injected through the process channel breaks the wall to perfect, generalizing success. When the required behavior is never sampled, no on-policy reward can induce it; imitation must seed what reward then grows—the same reachability wall the SWE probe hit from the other side, and the R1-Zero (discoverable) versus R1 (demonstration cold-start) distinction in miniature.

A.5 Learning where the outcome is dead: real bug-fixing at zero success

The variance account predicts that when a group is all-failing—outcome variance zero, outcome gradient dead—a process signal is the *only* thing that can move the policy. We check this directly on real software repair, a domain where the path-vs-outcome gap is well documented: in a controlled industry study, AI coding agents reach high functional correctness yet score far lower—and almost at random—on reliability, maintainability, and not breaking previously working functionality [Hong, 2026], the engineering discipline that decides whether a patch is deployable. Here an 8B agent is far below the task's reach, which makes it an ideal test of the dead-gradient regime. We train on SWE-smith [Yang et al., 2025] instances (a repo container, a hidden `FAIL_TO_PASS` test as the oracle), with the agent issuing bash commands to find and fix the bug; the process channel is the coding-discipline signal of §3 (a discharge for running tests and making a verifiable edit, a penalty for re-running a failed command or editing without testing). At this scale the agent solves essentially *zero* instances under either signal, so outcome-only RL trains on an all-fail distribution throughout—the cleanest

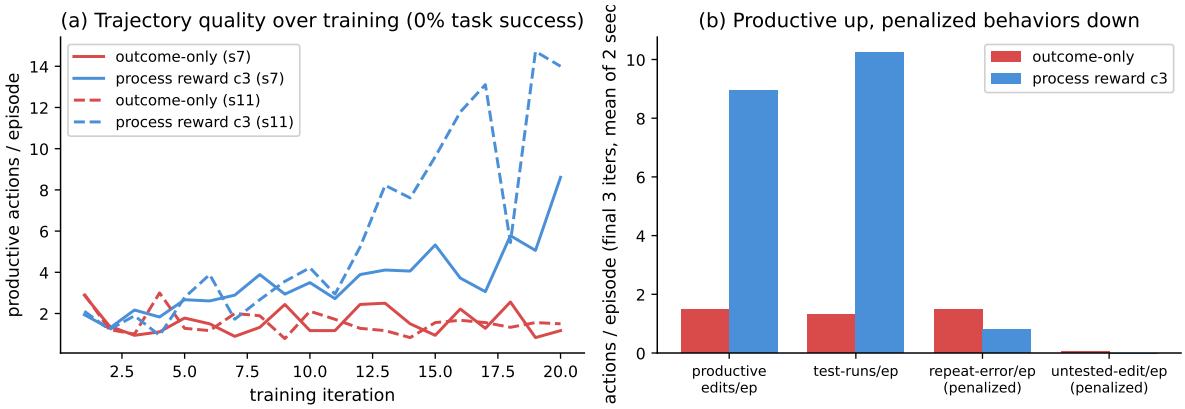

Figure 17. The process reward learns where the outcome is dead (SWE-smith, 8B, ≈0% solve). *(a)* Productive actions per episode over training, two seeds: outcome-only is flat (the all-fail gradient is dead), while the process reward rises steadily. *(b)* Final-3 trajectory quality (mean of two seeds): productive edits and test-runs increase several-fold under the process reward, while penalized behaviors (repeat-errors, untested edits) fall. Task success is zero for both arms throughout—this is a behavioral/mechanistic result, not a success result.

possible test of the dead-gradient regime. This is a *different* signal than the progress *potential* of §4: on this same software-repair domain the test-fraction potential is vacuous (Figure 8, left), because zero tests ever pass and so no intermediate value is reached. The penalized and discharged behaviors here—running a test, editing a file, re-running a failed command—are always reachable, so the penalty channel carries within-group variance exactly where the potential cannot. The two software-repair results are consistent: the reachability gate closes on the potential and stays open on the penalty.

The result is a sharp dissociation (Figure 17, two seeds). Outcome-only leaves the trajectory *flat*: productive actions per episode sit at ≈1.5 from the first iteration to the last, essentially identical across seeds (1.5 and 1.5)—the dead gradient changes nothing. The process reward instead drives a steadily *rising* trajectory: productive actions climb over training to 6.5 and 11.4 (final-3, the two seeds) and test-runs to 7.6 and 12.9, a four-to-tenfold increase, while the penalized behaviors fall (repeat-errors and untested edits toward zero). The agent learns to investigate, test, and edit like a disciplined engineer—from a signal available on every failed episode—exactly where the outcome reward is inert.

Two honest caveats scope the claim. First, the discharge *rewards* test-running and productive edits, so part of their rise is by construction; the non-circular evidence is that the *penalized* behaviors (which the discharge does not touch) also fall, and that the outcome baseline is perfectly flat—the effect is the process gradient, not the task. Second, this improved discipline did *not* convert to task success in our budget (both arms remain at zero): 8B is simply too weak to close these bugs. The claim is therefore mechanistic and behavioral—the process channel produces learning, and better verifiable engineering discipline, precisely in the all-fail regime where outcome-only RL has no gradient at all—and a plausible precursor to success at capability, not a demonstration of success itself.

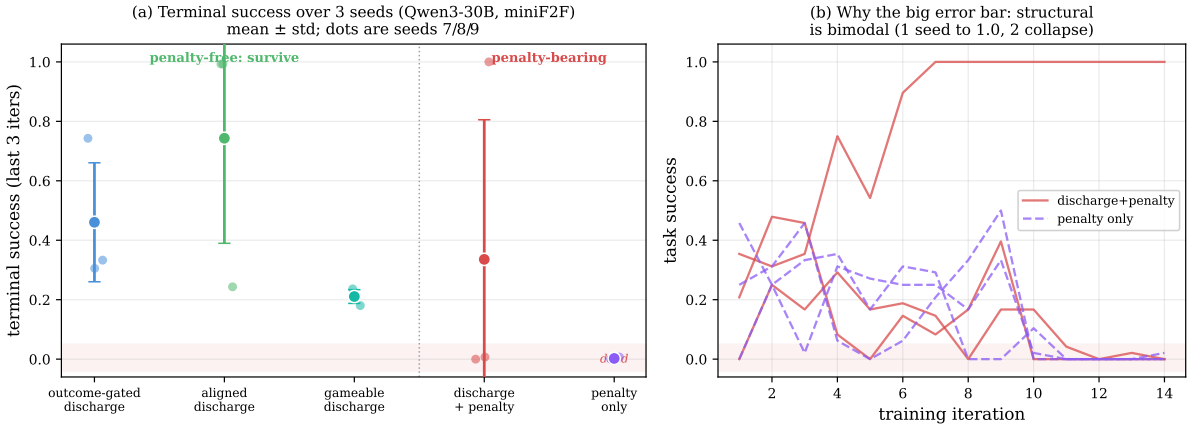

Figure 18. Which signals survive (Qwen3-30B, miniF2F, 3 seeds). *(a)* Terminal success (last-3-iteration mean) for five signals, mean ± std with seed dots. The three penalty-free signals sit well above the dead zone on every seed; the pure penalty is reliably dead; the discharge+penalty arm has by far the largest error bar. *(b)* That arm is *bimodal*—collapsing on two seeds, rescued to 1.0 on one—whereas the pure penalty collapses on all three.

B Boundaries: When the Penalized Target Is Not Verifiable

Design rule 4 (§3.1) requires the penalized (and discharged) target to be reachable and *un-gameable*. This appendix gives the experiments behind that rule: what happens when a penalty's target is misaligned with the outcome, and when it is a *learned* proxy rather than a verifiable predicate.

B.1 An un-gameability sweep: which signals survive

We construct five process signals for Lean theorem proving on miniF2F [Zheng et al., 2022], spanning aligned to misaligned, pre-register each one's cheapest gaming policy, and train Qwen3-30B-A3B [Team, 2025] with each (three seeds, everything else fixed; Figure 18). Survival tracks admissibility. The *penalty-free* signals reliably survive on every seed: a gameable "any-valid-tactic" credit, an aligned goal-progress discharge, and that credit *gated on the eventual outcome* (un-gameable by construction, the most consistent survivor). The *pure penalty*—an errored-tactic penalty with no discharge and no outcome reward—reliably **collapses to essentially zero on every seed**: its cheapest maximizer, avoid errors by not attempting, is precisely the *inaction trap* of design rule 2, and with no positive signal to escape it the policy dies every time. This is the paper's cleanest evidence that a lone penalty cannot drive learning. The revealing case is *penalty-plus-discharge*: across three seeds it is bimodal (collapses on two, rescued to perfect on the third), the largest variance of any arm—adding a discharge to a misaligned penalty makes survival *possible* but not reliable. The robust reading: penalty-free progress signals survive, a pure penalty reliably collapses, and outcome-gating is the most consistent survivor. Precise magnitudes remain noisy at 30B; we report the survival pattern.

B.2 Task intent is not a verifiable rule

The hardest case for a penalty is a task whose difficulty is not procedural. On τ-bench-style airline customer service [Yao et al., 2024] the reward turns on *semantic* policy adherence (authorization, refund eligibility, confirmation) rather than structural ordering, so no verifiable

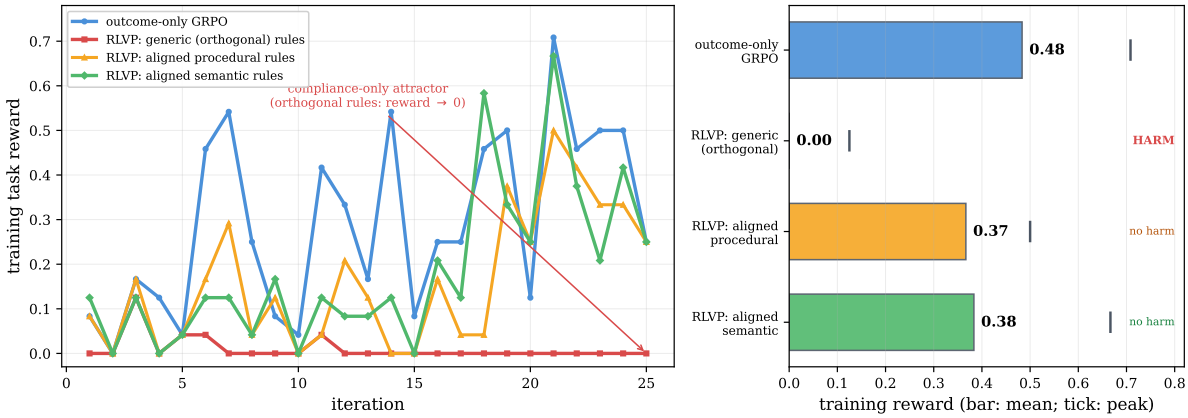

Figure 19. The coverage gradient on τ-bench airline. *Left:* training reward for four signals. Outcome-only learns the benchmark; generic rules orthogonal to the reward collapse into the inaction trap; policy-derived procedural and verifiable-semantic rules (with early annealing) do not. *Right:* mean reward per tier (bars) with peak (ticks). Misaligned rules harm; aligned rules remove the harm and nearly reach outcome-only's peak, but neither beats it. The residual gap is intent.

rule is finer than the outcome: what is missing is intent, which is the reward itself. Three tiers make this concrete (Figure 19). *Generic* structural rules—the read-before-write hygiene that works on chained tasks—are now orthogonal to the reward and actively *harm*: the cheapest way never to violate "confirm before calling" is to not place the risky calls at all, and the policy collapses into the inaction trap within a handful of iterations. *Policy-derived procedural* rules (fetch the user id and look up the reservation before modifying it), outcome-instrumental by construction and paired with early annealing, remove the harm, because a correct modification requires the looked-up state so the discharge pulls toward the productive workflow. *Verifiable semantic* rules (do not modify a basic-economy reservation; do not use a payment method absent from the profile) prune failure-bound actions and lift the best iterations nearly to outcome-only's peak. But neither aligned tier *beats* outcome-only on average (Figure 20): verifiable rules express policy *validity*, while the residual difficulty is *which* permissible change this user wants—and a rule encoding that would just be the task specification.

B.3 A learned critic relocates the hack

If verifiable rules cannot reach intent, why not let the policy judge itself, in the lineage of self-rewarding LMs and RLAIF [Yuan et al., 2024b, Lee et al., 2023, Bai et al., 2022, Zheng et al., 2023]? We use the critic as the *same model* as the policy (no larger judge), so any signal is genuine on-policy self-reflection. Across two axes—whether a verifiable rule is cheaply specifiable, and whether the on-policy critic can detect the violation (Figure 21)—the learned critic is a poor training reward in every quadrant. As a detector, blind self-critique flags surface-evident violations but is essentially blind to stateful-bookkeeping norms (did it read *this* file before overwriting it) even when handed the rules, and the blind spot is structural (per-rule recall stays near zero as the critic scales). As a reward in the all-fail regime (Table 3), a deterministic rule with identical credit structure is decisive on every seed while the self-critic—*with perfect recall* against the rule oracle—is inert; a *frozen* critic fails identically, isolating the cause as the critic's $\sim$6% false-positive rate, not its non-stationarity. At the intent frontier (Table 4) the self-critic is a useful *offline* detector yet collapses as a training reward. Defining the penalized

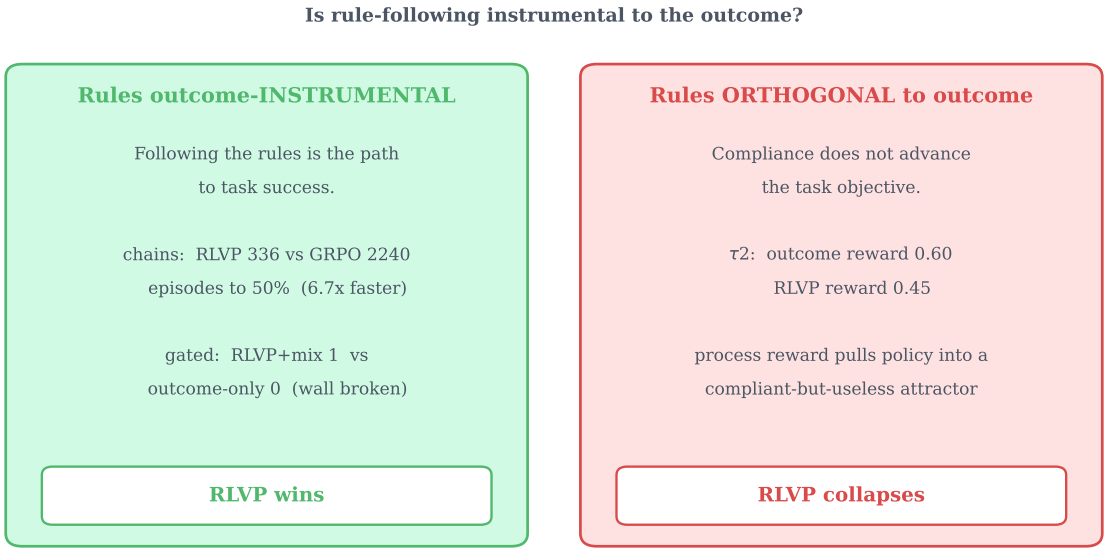

Figure 20. The boundary on one axis: are the rules outcome-instrumental? When the verifiable rules are aligned with what success requires (chained and gated tasks: the workflow *is* the task), the process channel converts compliance into large outcome gains. When orthogonal to the reward (τ-bench, generic rules), the same machinery optimizes compliance into a degenerate policy.

Figure 21. When a verifiable rule beats a same-model self-critic, and when neither trains. Axes: whether a verifiable rule is cheaply specifiable, and whether the on-policy critic can detect the violation. The learned critic is a poor training reward in every quadrant; its only robust use is offline diagnosis at the intent frontier.

target by a learned proxy relocates the credit-assignment problem into the proxy, which the policy then games—the mechanism behind design rule 4. Verifiability is what makes the dense channel safe to optimize.

Table 3. A deterministic rule beats a same-model self-critic as a dense reward, in the all-fail regime (consumer-service domain, 1.7B, three seeds, late-training mean ± std). Rule and critics share the identical penalty-only credit structure, and the critic additionally has *perfect recall* against the rule oracle. The rule is decisive every seed, while the critics are inert. The *frozen* (stationary) critic fails like the live one, isolating the cause as the critic's ∼6% false-positive imprecision rather than non-stationarity.

process reward	detection (P/R)	violations/episode	task success
rule penalty (deterministic)	1.00/1.00	$\mathbf{0.38 \pm 0.44}$	$\mathbf{0.33 \pm 0.47}$
self-critic, live (co-evolving)	0.94/1.00	0.94 ± 0.05	0.00
self-critic, frozen (stationary)	0.94/1.00	0.93 ± 0.02	0.00

Table 4. Self-critique is a usable intent *detector* but a failed intent *reward* (τ-bench airline, Qwen3-4B). *Left:* as a failure predictor on rule-clean (intent) failures the self-critic far exceeds the verifiable semantic rules (3 rollout seeds). *Right:* as the dense training reward the same self-critique collapses, while outcome-only and the rules both learn (training reward, early→late, 20 iterations).

offline: failure prediction		*trained as reward*	
channel	F1	channel	reward (early→late)
semantic rules	0.23 ± 0.06	outcome-only	$0.09 \rightarrow \mathbf{0.50}$
blind self-critique	$\mathbf{0.63 \pm 0.02}$	semantic rules	$0.10 \rightarrow 0.35$
		blind self-critique	$0.01 \rightarrow \mathbf{0.00}$

C Setting-by-Setting Summary

Table 5 collects the five settings studied across the paper against the three properties the variance account cares about—does a verifiable signal finer than the outcome exist, is it un-gameable, and are its intermediate values reachable—together with the observed outcome. Help appears only when a reachable, un-gameable signal is available; a penalty on always-sampled bad actions satisfies this on every row where it applies, while a progress potential does so only where partial progress is reachable.

Table 5. The five settings studied, summarized against the variance account. Each row is a domain. The three middle columns are the conditions the account cares about: does a verifiable signal finer than the outcome exist, is it un-gameable, and are its intermediate values reachable. "✓/✗" marks the condition that decides the row. A dense signal helps only when all three hold; this table organizes the settings rather than predicting them.

Setting	Domain / model	Finer Φ?	Un-game?	Reach?	Verdict	Observed
Theorem proving (progress)	Lean miniF2F 30B, synth. 4B	✓	✓	✓	**help**	dead updates $\to$ 0, dense gradient
System admin (harm)	TerminalBench 4B	✓	✓	✓	**help** (harm axis)	$\sim$6$\times$ fewer violations, equal success
Lean signal sweep (controlled)	miniF2F 30B	varies	**varies**	✓	per arm	pure penalty dies, penalty-free survive
Customer service (intent)	τ-bench airline 4B	✗	—	—	**no help**	rules match, never beat outcome
Software repair	SWE-bench 30B	1/3 only	—	✗	**no help**	0% solve, all rollouts Φ=0